

Git 使用手册

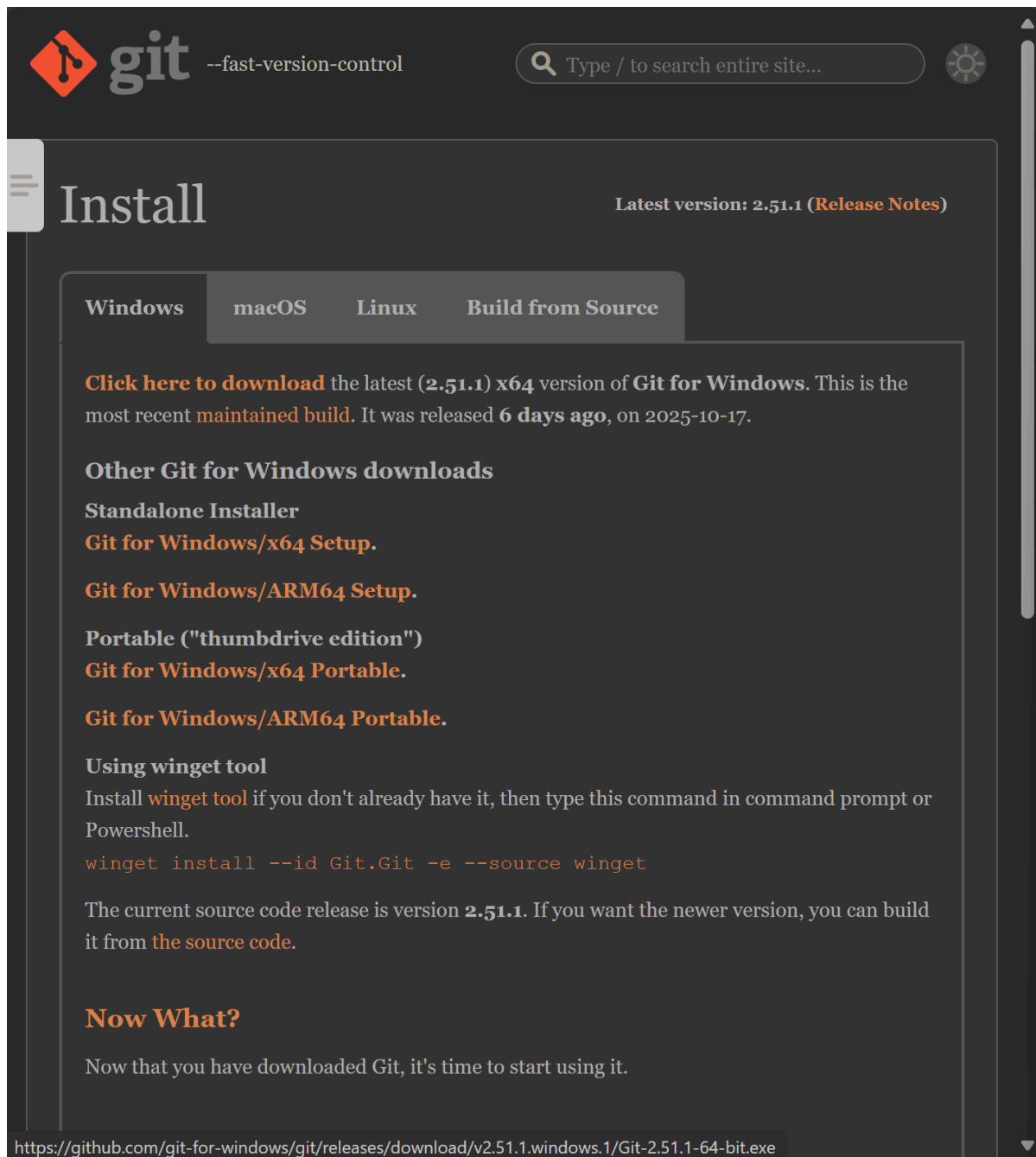
① Note

这个文档可能有点罗里吧嗦，可以借助一下目录 $q(\geq \nabla \leq q)$

Git 的安装

官网：

[Git - Install for Windows](#)



git --fast-version-control

Type / to search entire site...

Install

Latest version: 2.51.1 ([Release Notes](#))

Windows macOS Linux Build from Source

Click here to download the latest (**2.51.1**) **x64** version of **Git for Windows**. This is the most recent **maintained build**. It was released **6 days ago**, on 2025-10-17.

Other Git for Windows downloads

Standalone Installer
Git for Windows/x64 Setup.
Git for Windows/ARM64 Setup.

Portable ("thumbdrive edition")
Git for Windows/x64 Portable.
Git for Windows/ARM64 Portable.

Using winget tool
Install **winget tool** if you don't already have it, then type this command in command prompt or Powershell.
`winget install --id Git.Git -e --source winget`

The current source code release is version **2.51.1**. If you want the newer version, you can build it from **the source code**.

Now What?

Now that you have downloaded Git, it's time to start using it.

<https://github.com/git-for-windows/git/releases/download/v2.51.1.windows.1/Git-2.51.1-64-bit.exe>

一般可以选择第一项： `Click here to download` 或 `Git for windows/x64 Setup.`

安装

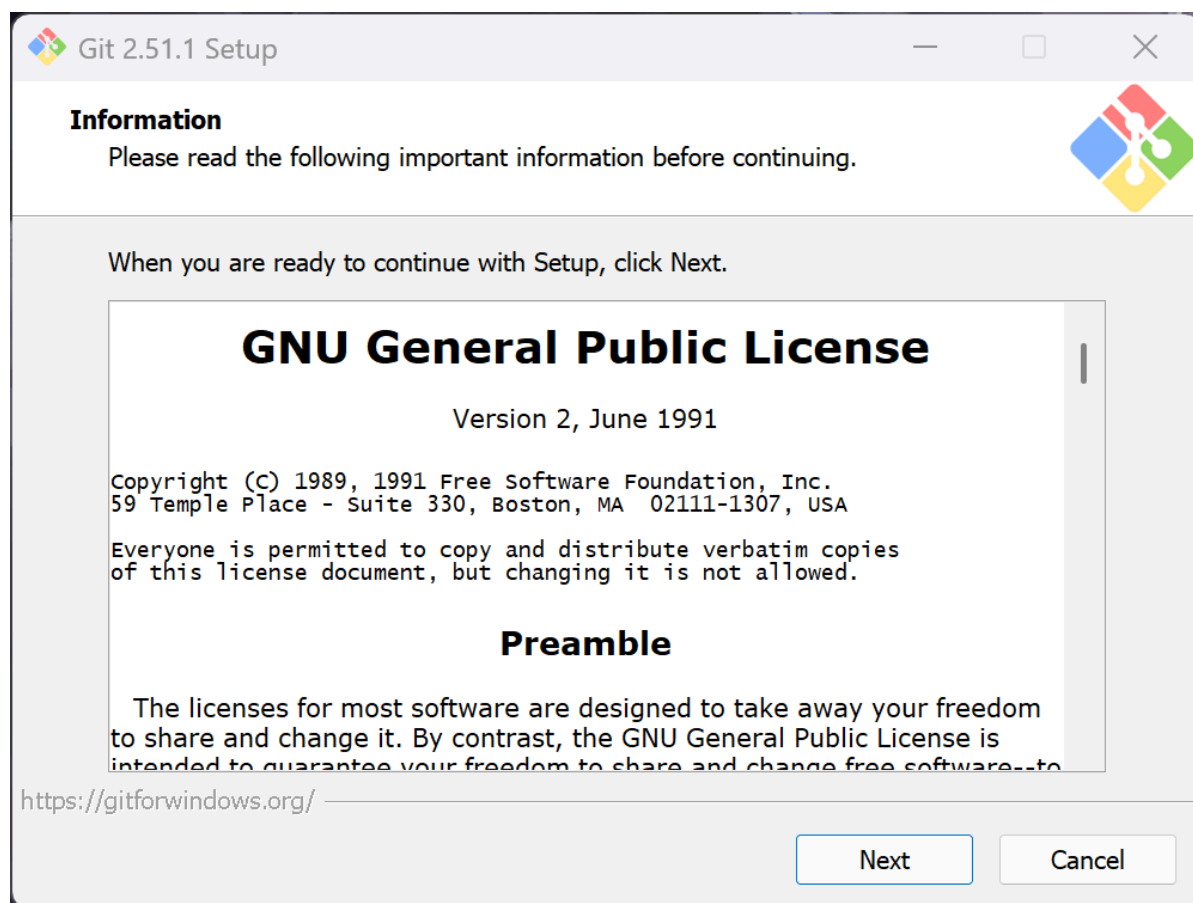
总纲

其实没啥，大部分默认就行。小建议：可以改一下默认的文本编辑器，比如改成VScode，不然很痛苦！（血的教训）

详情

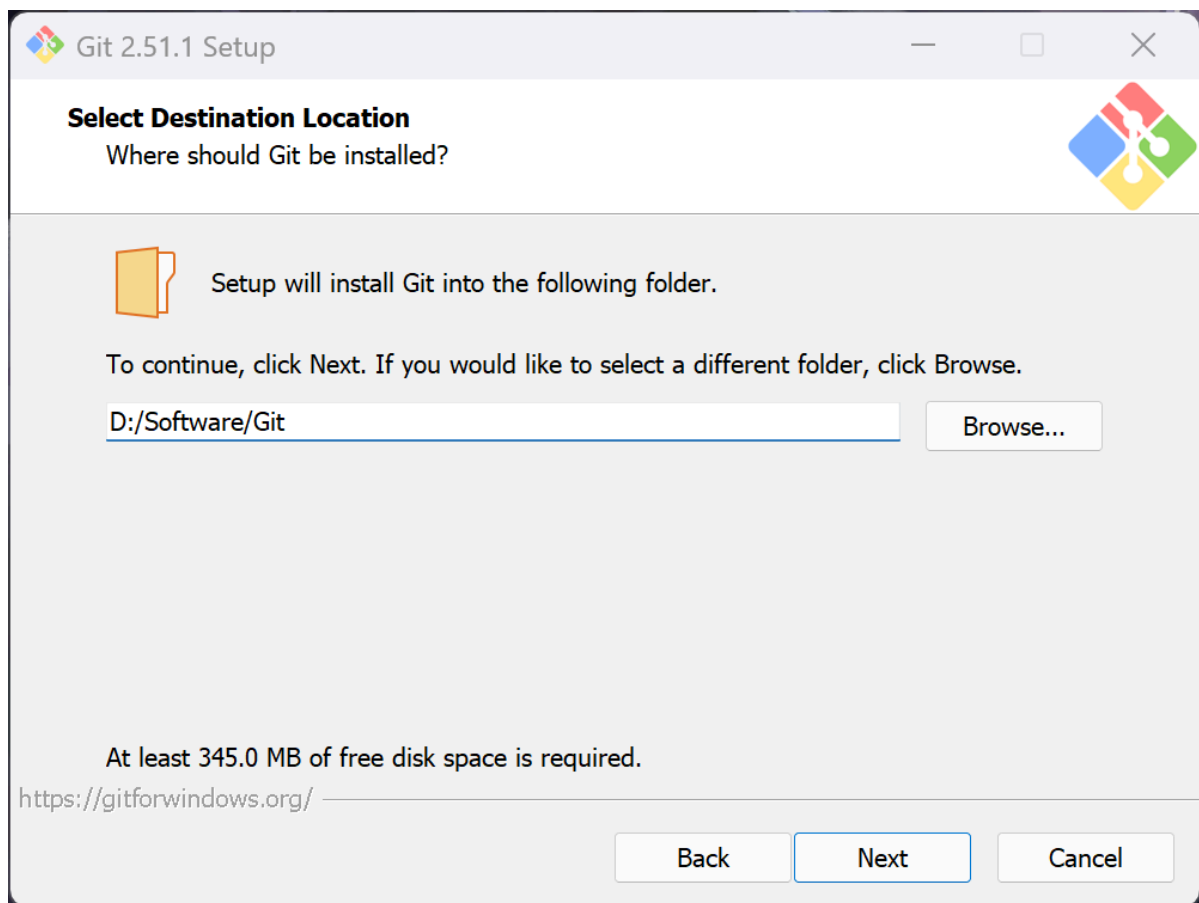
第一页

就是读协议嘛



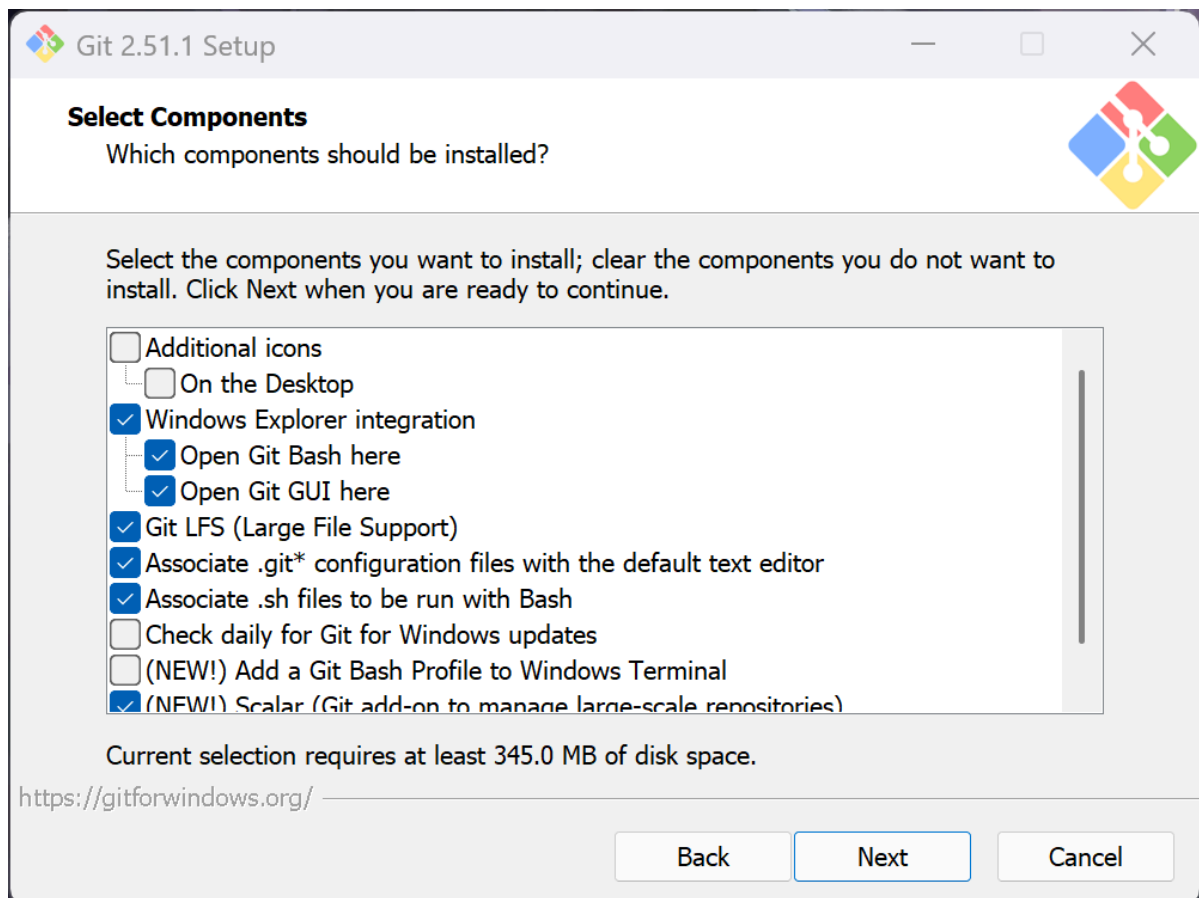
第二页

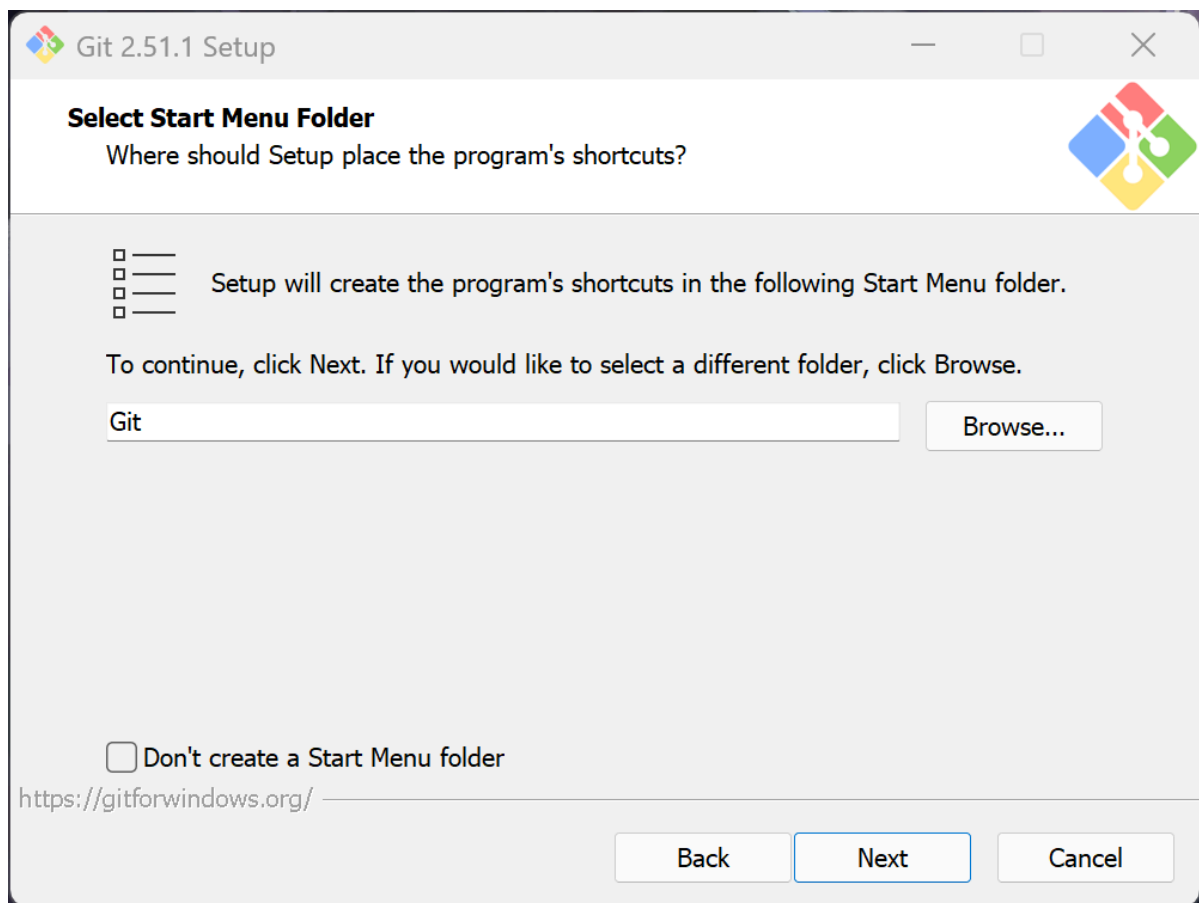
就是安装路径，可以自己挑一个习惯的



第三、四页

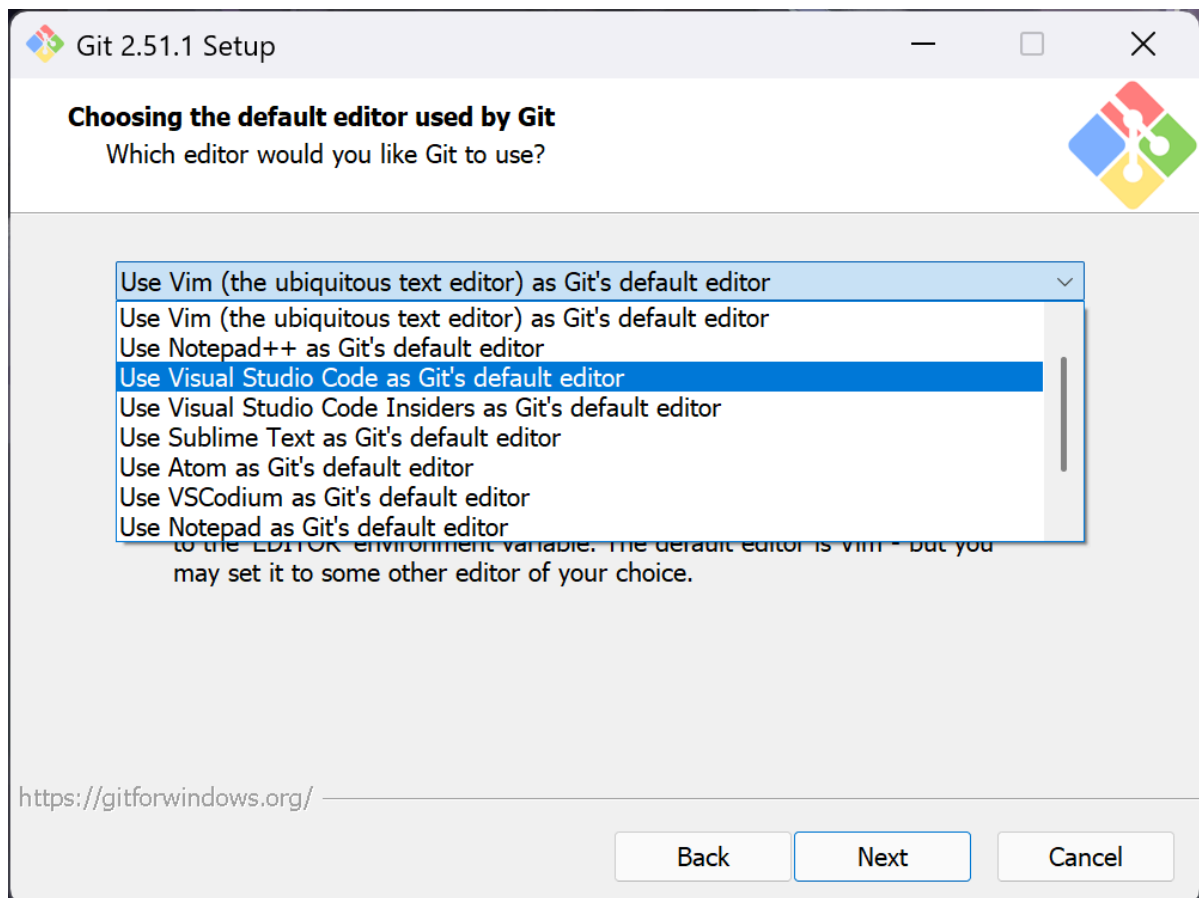
默认就行，我看了里面默认就开启了Git LFS，如果实在不放心可以在第三页 **Select Components** 仔细看看，有没有勾上 **Git LFS(Large File Support)**

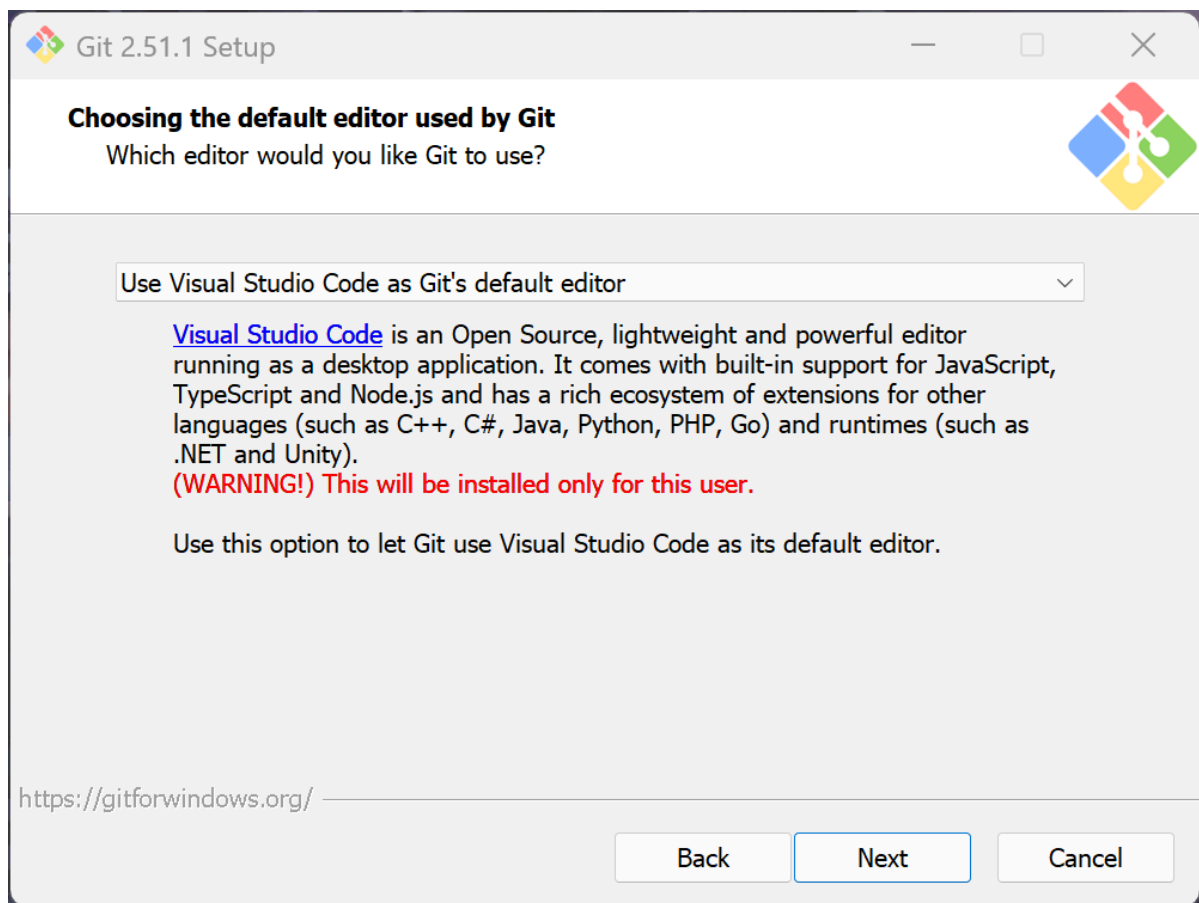




第五页!

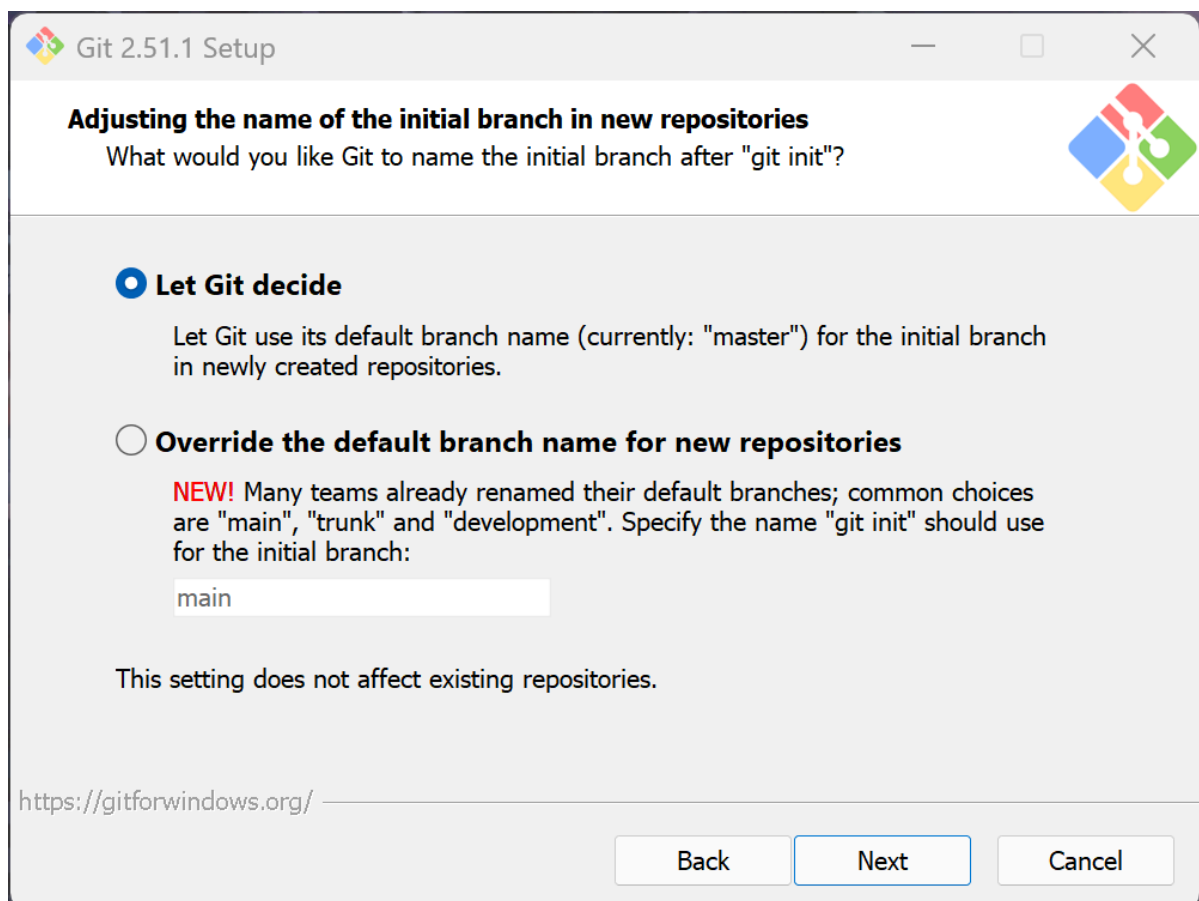
强烈建议不要用它默认的文字编辑器! 太**难用了bro!!! 可以选 `VScode` 或是 `Notepad (记事本)` , 看个人喜好啦!

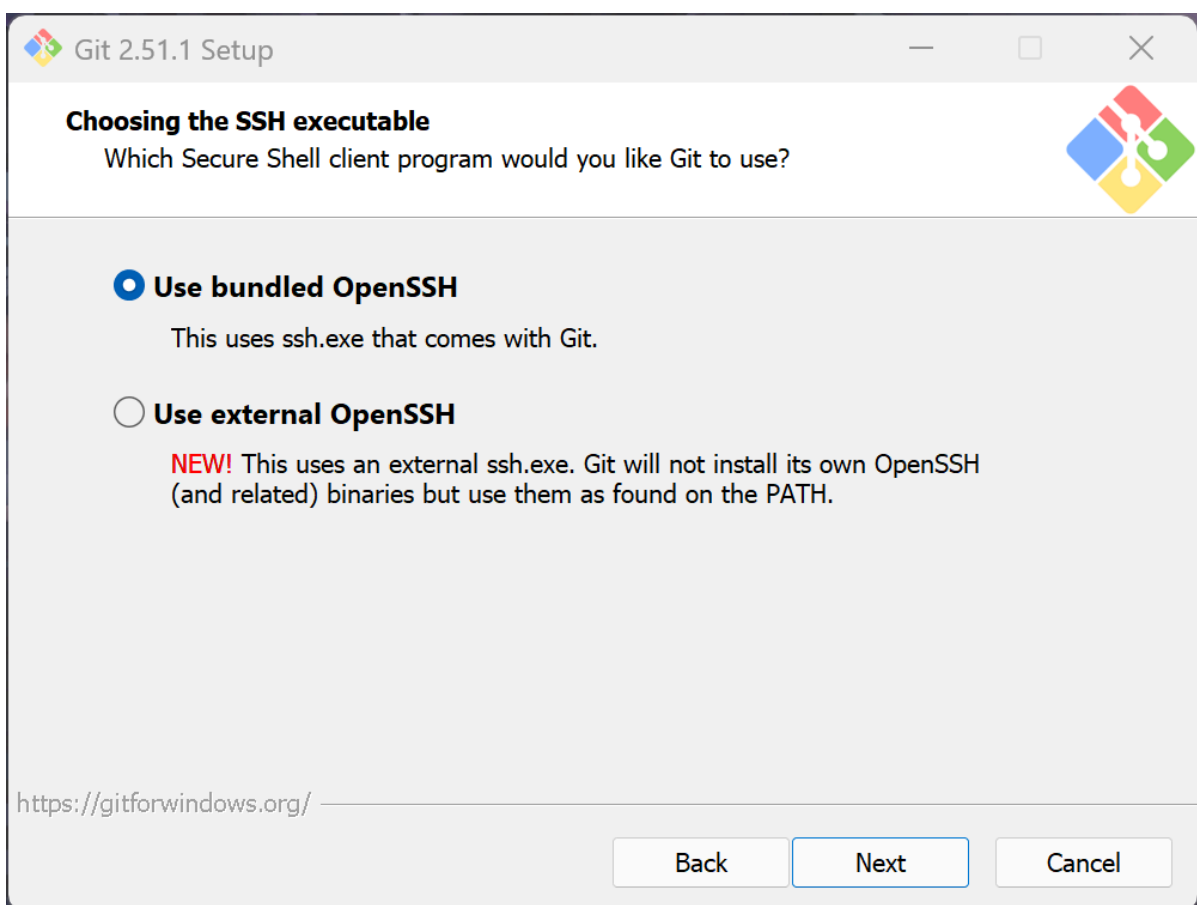
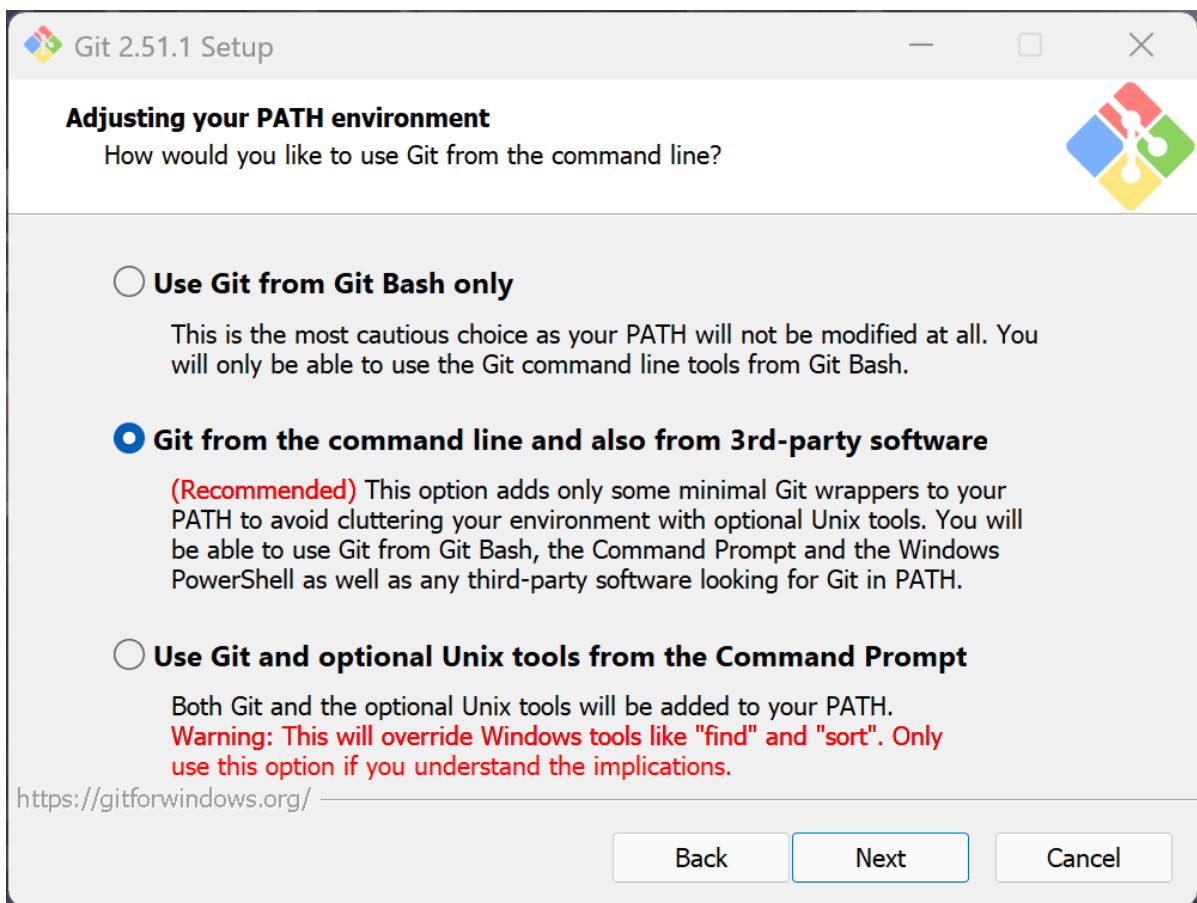


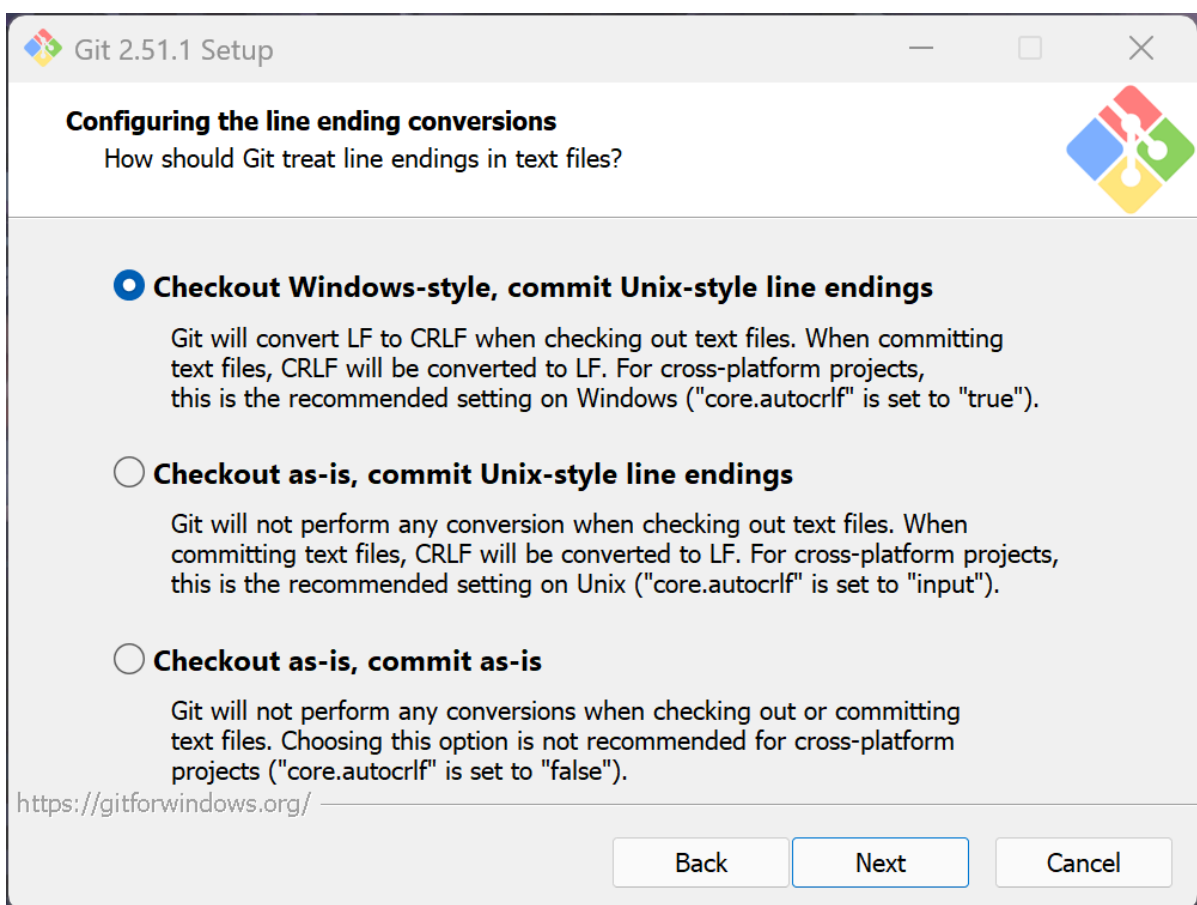
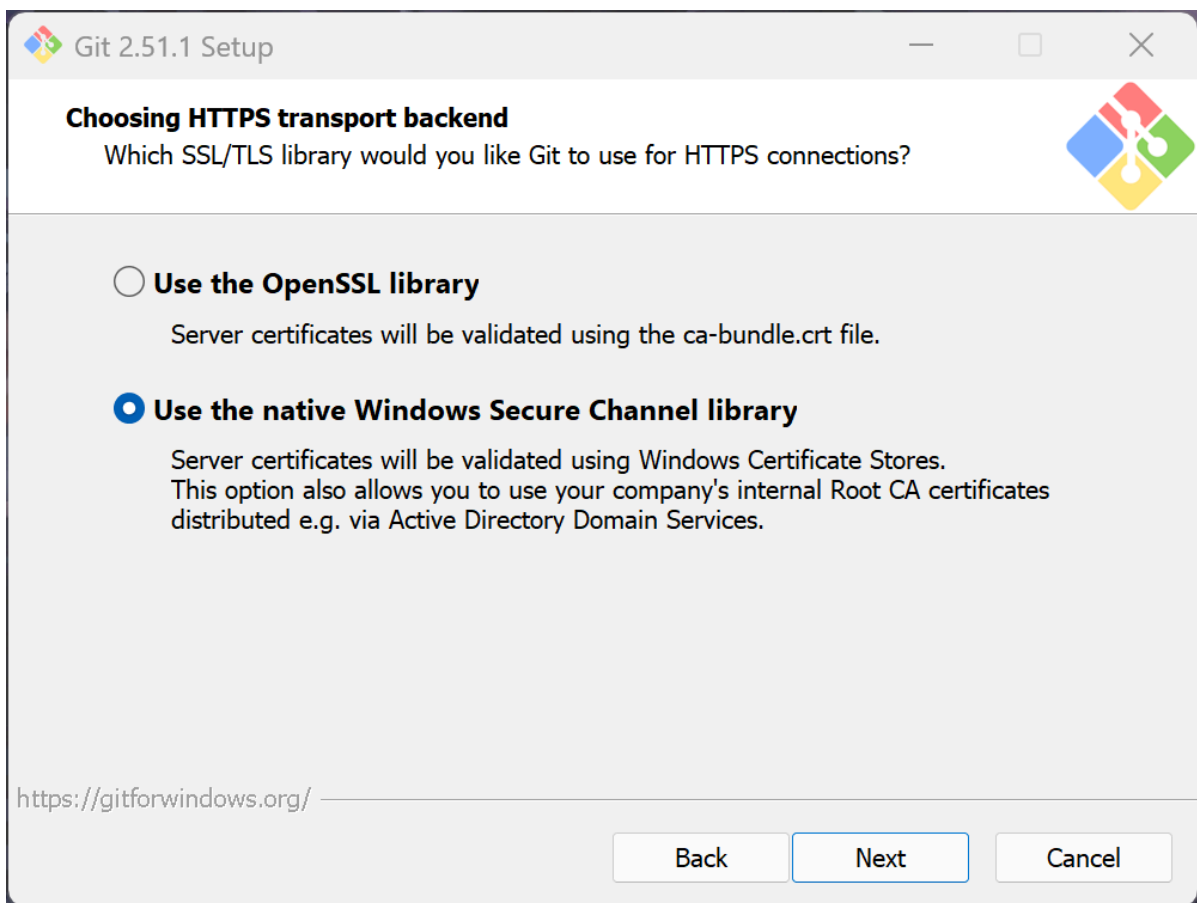


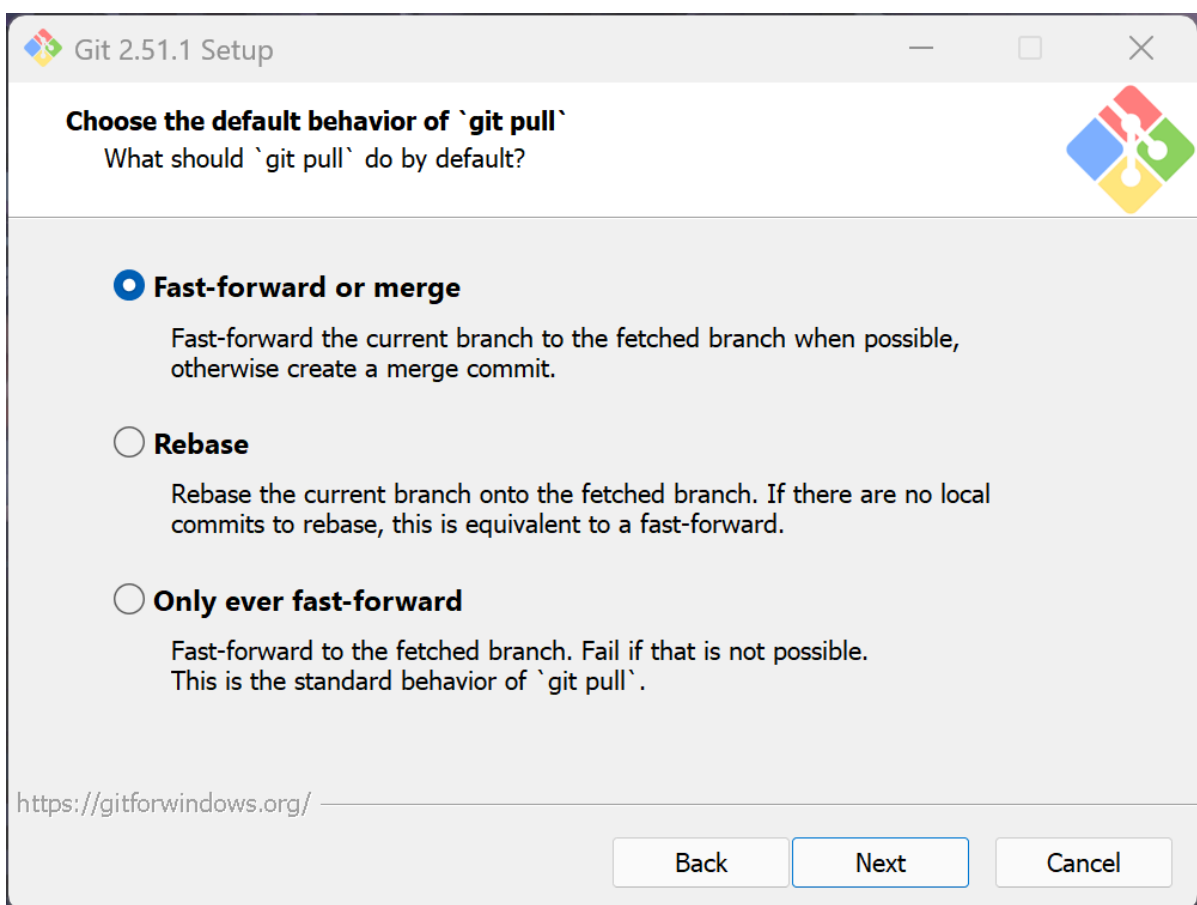
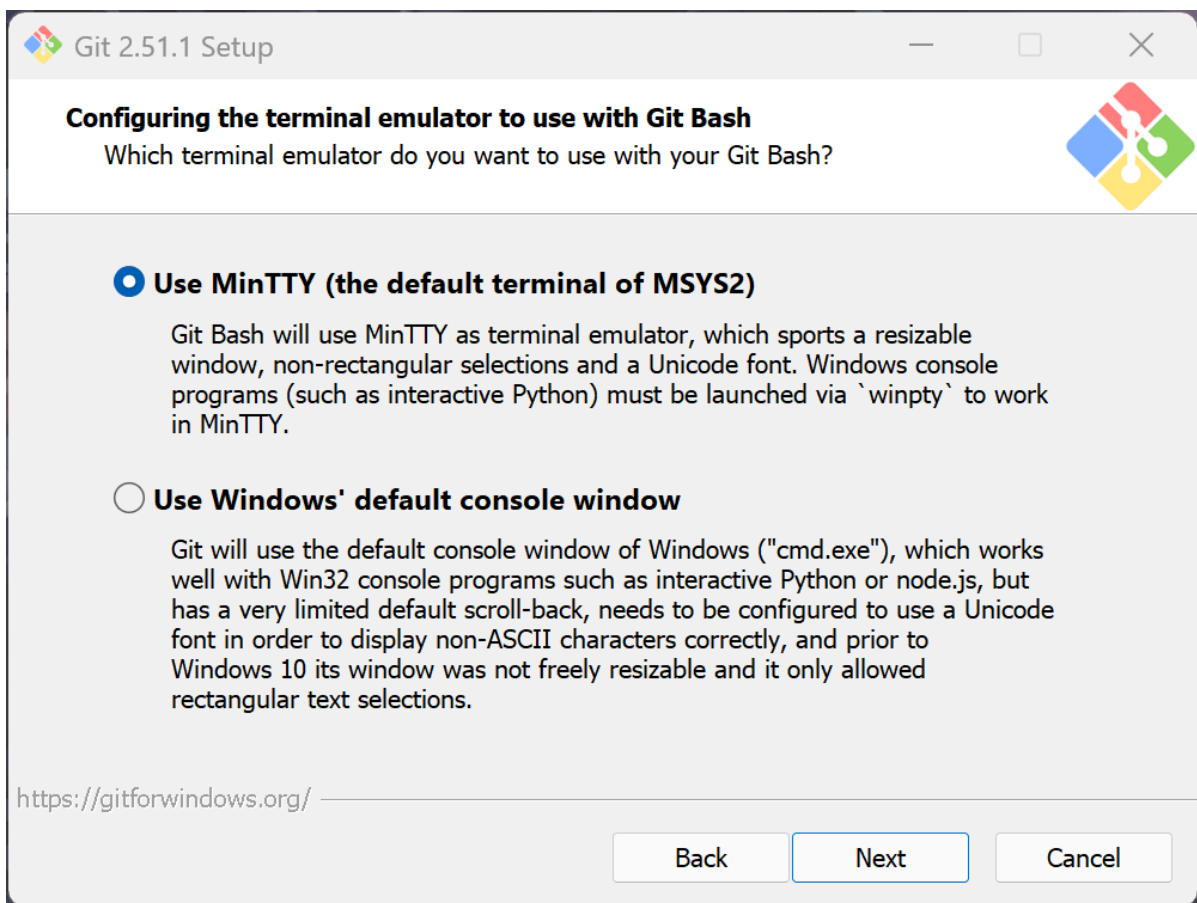
后面全部默认就行

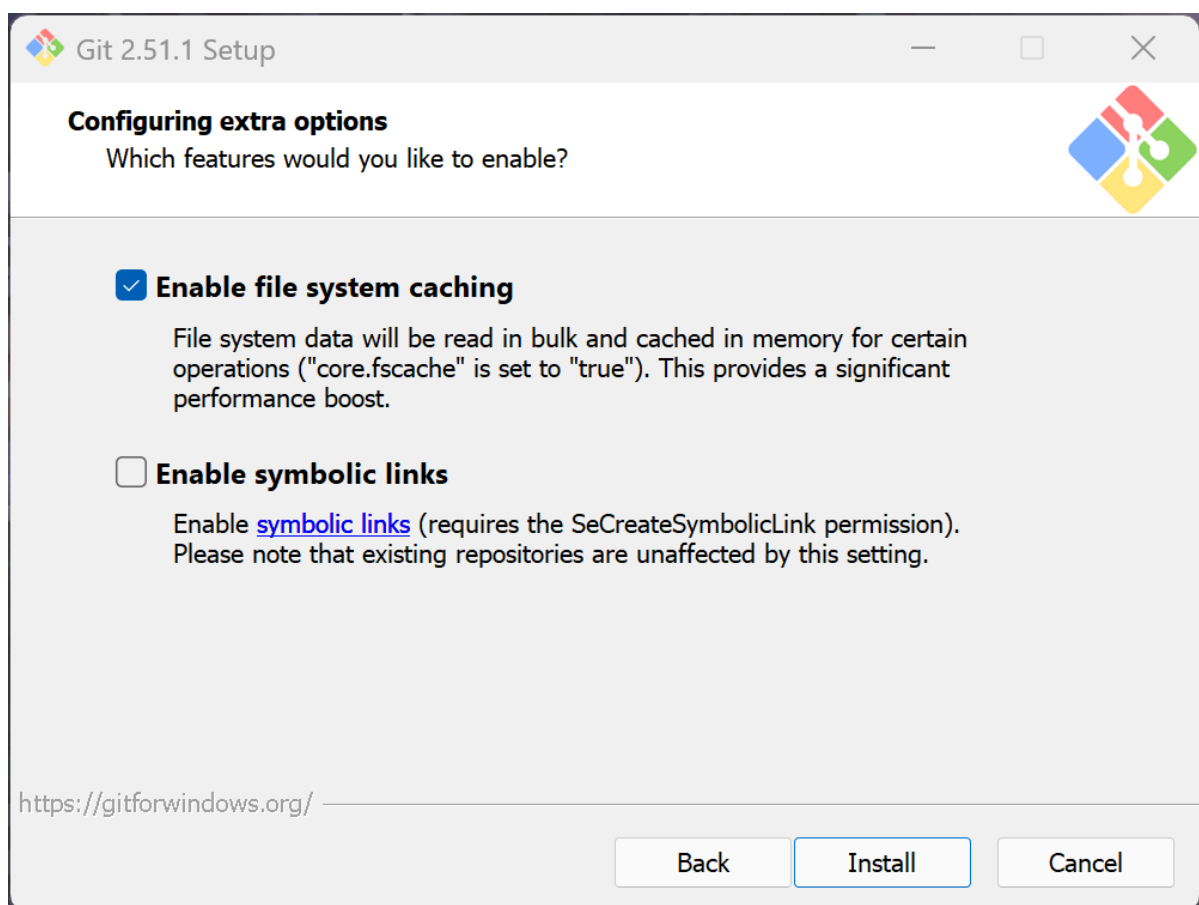
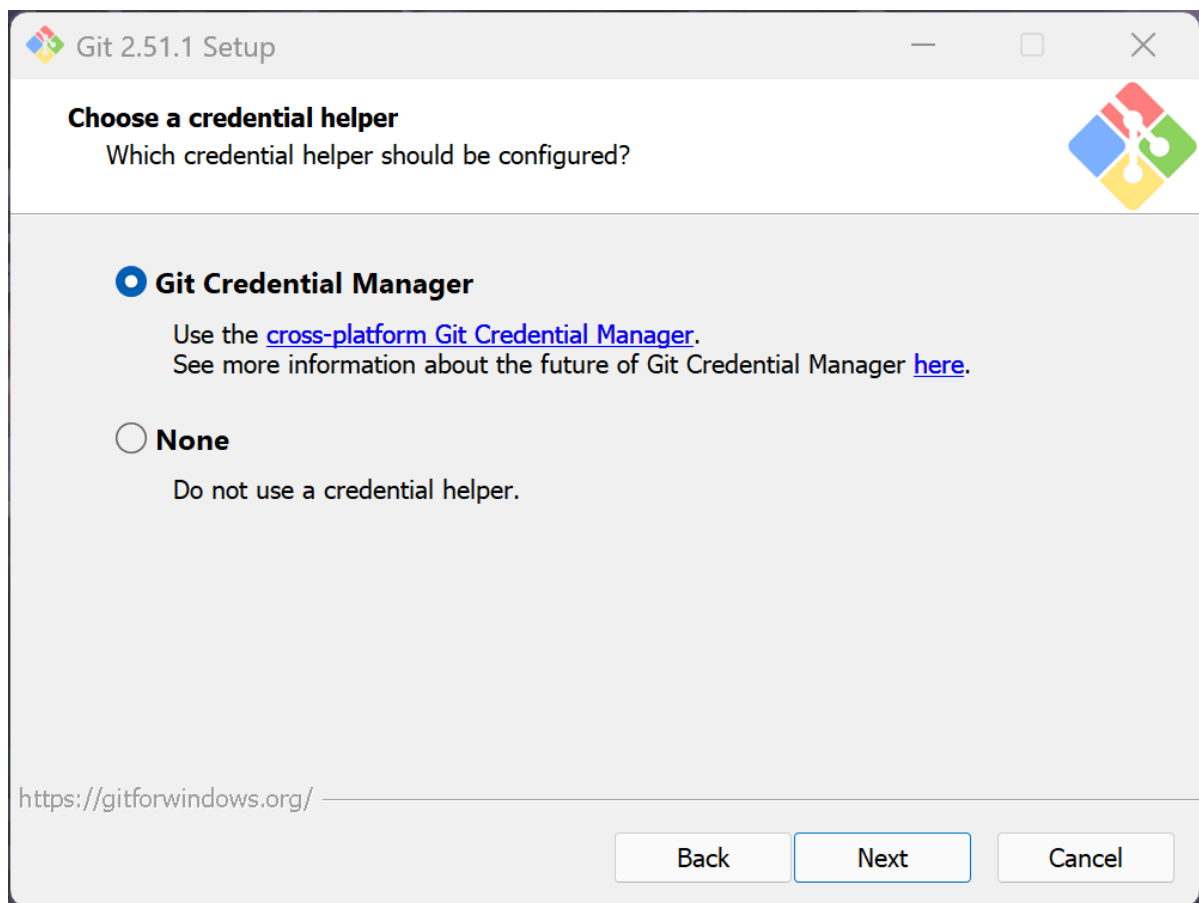
给你们晒个流程（其实也可以跳过不看）



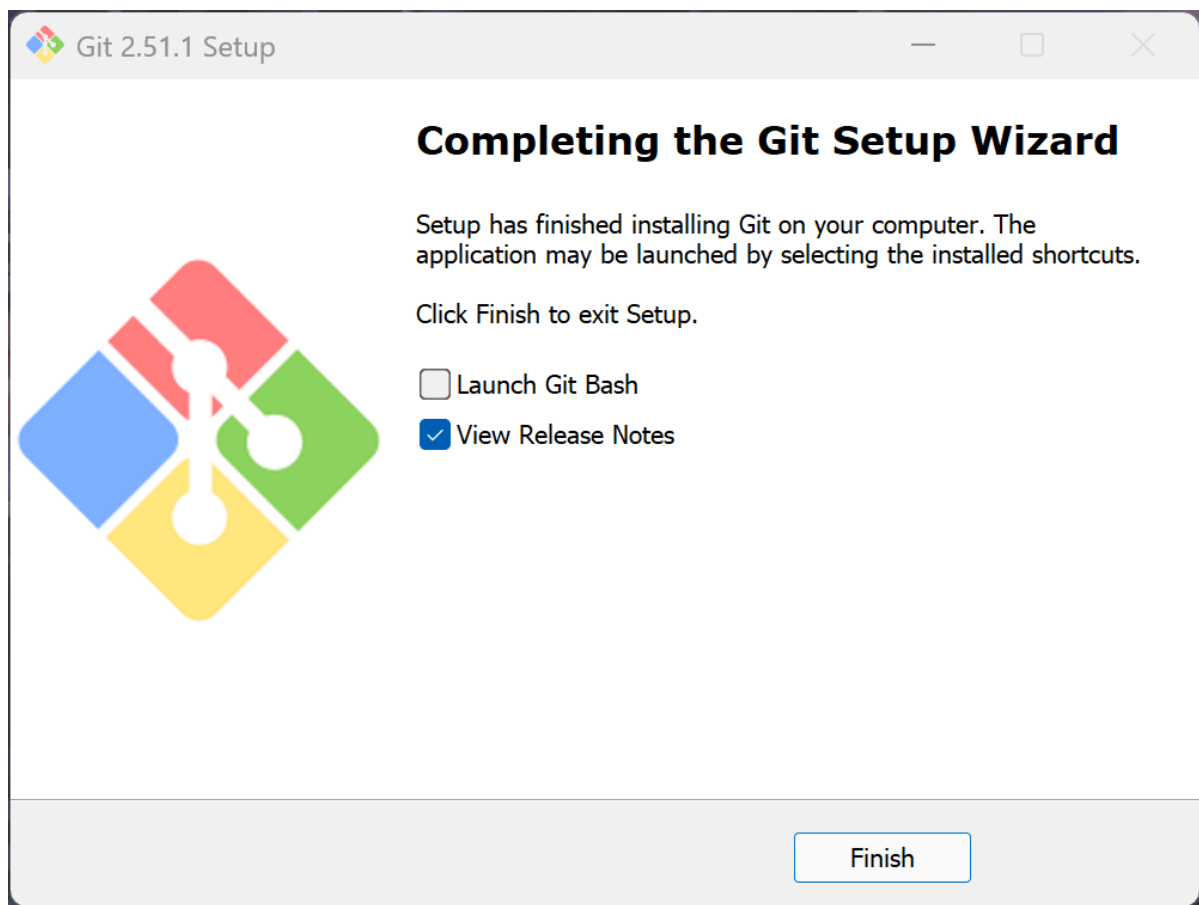






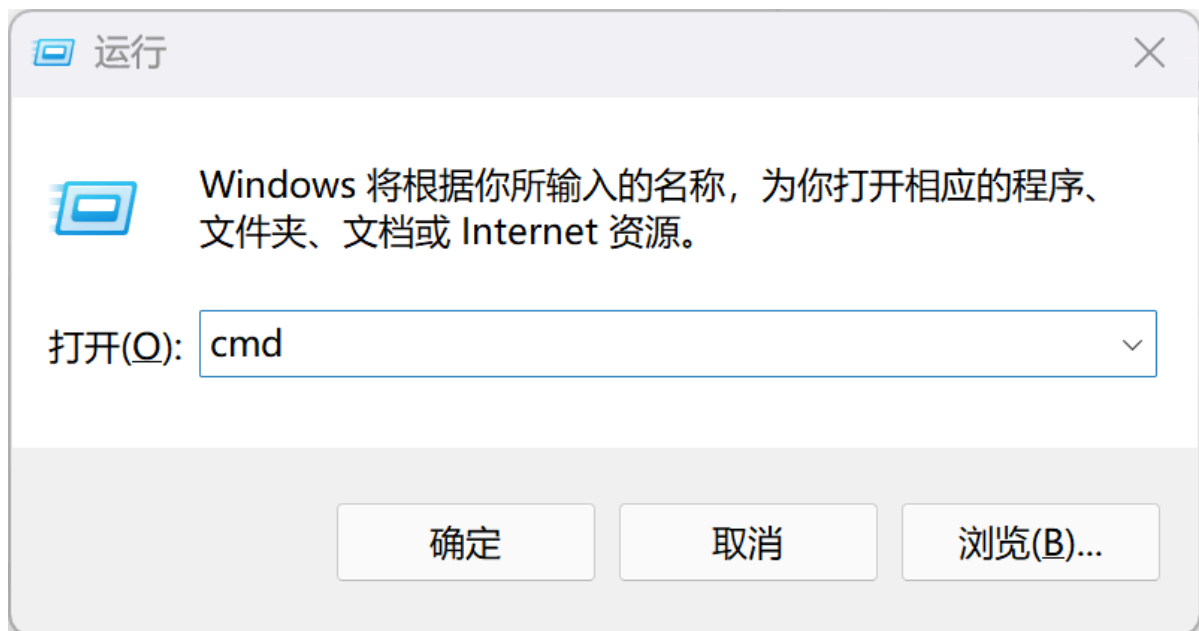


好了，安装完毕！（如下图）



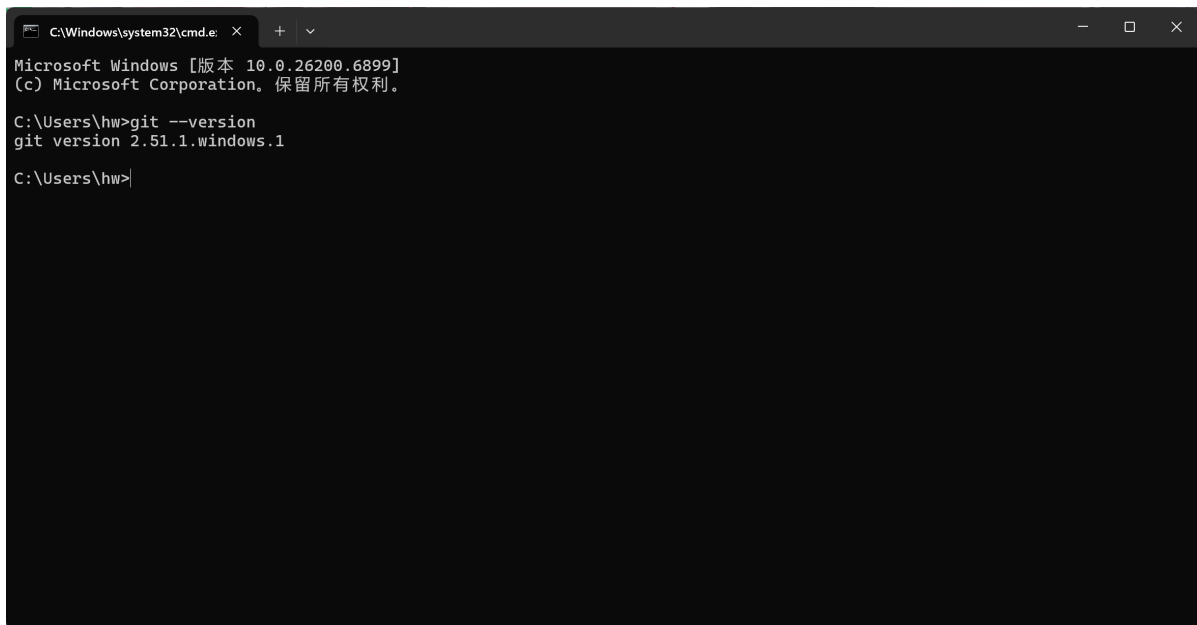
验证

键盘 `win + R` 调出运行，输入 `cmd`



然后就会弹出一个小黑框，这就是 命令提示符 (`cmd`)。输入：（注意有两个减号！）

```
git --version
```



```
C:\Windows\system32\cmd.e. X + v
Microsoft Windows [版本 10.0.26200.6899]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\hw>git --version
git version 2.51.1.windows.1

C:\Users\hw>|
```

能这样正常弹出版本号，就说明Git安装成功！

Git 的配置

加速器

知周所众，登录 Github非常滴看脸.....

(夹带私货系列) 这边有一个加速器推荐：**Watt Toolkit (原名Steam ++)**

[瓦特工具箱\(Steam++官网\) - Watt Toolkit](#)

别看它是Steam加速器，它附带Github加速！

至少我的电脑上，平常不开加速器的话，浏览器正常访问Github网页，可是本地Git就根本访问不了一点。。。。

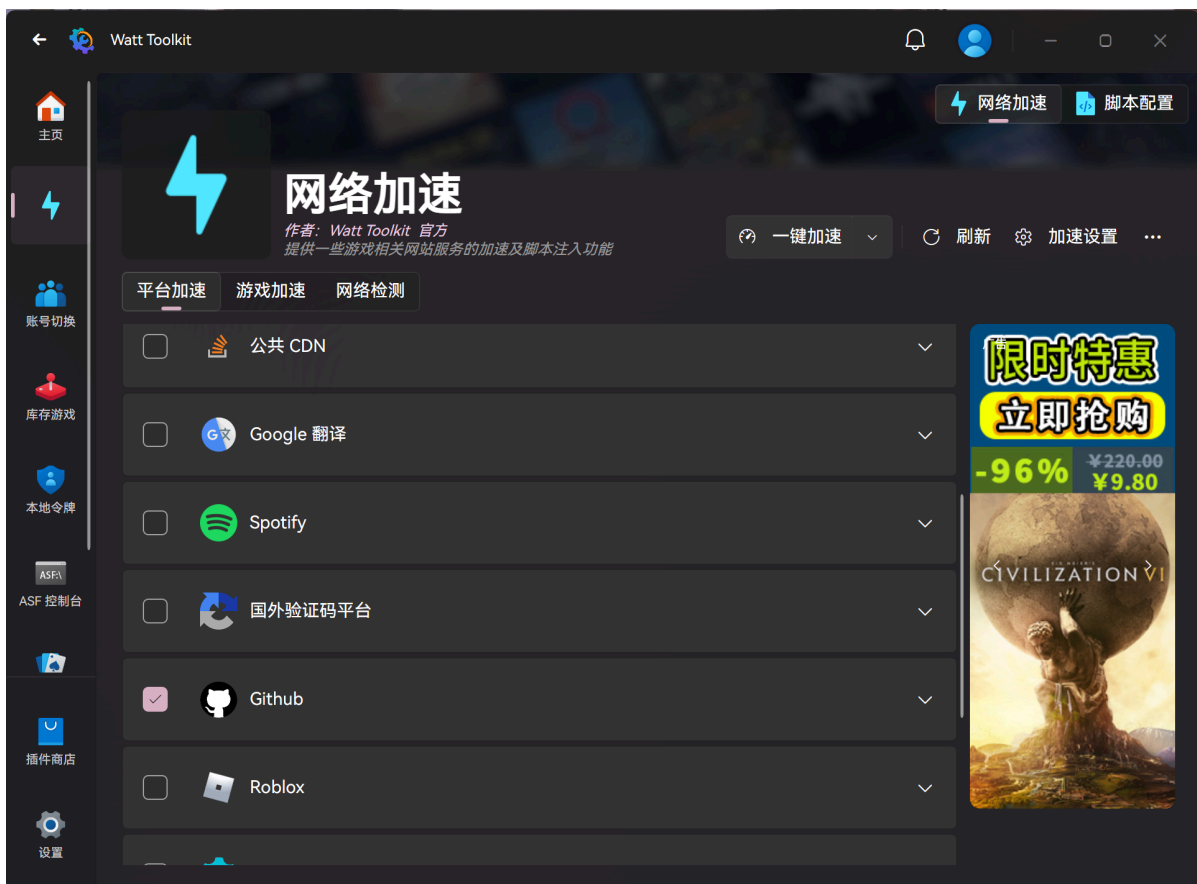
所以个人是建议提交代码的时候尽量开开加速器。

啊当然也可能是我的问题。

主页长这样：



点击 网络加速 ，往下滑就有 Github 了：



Github 账号

首先你要注册一个Github账号。

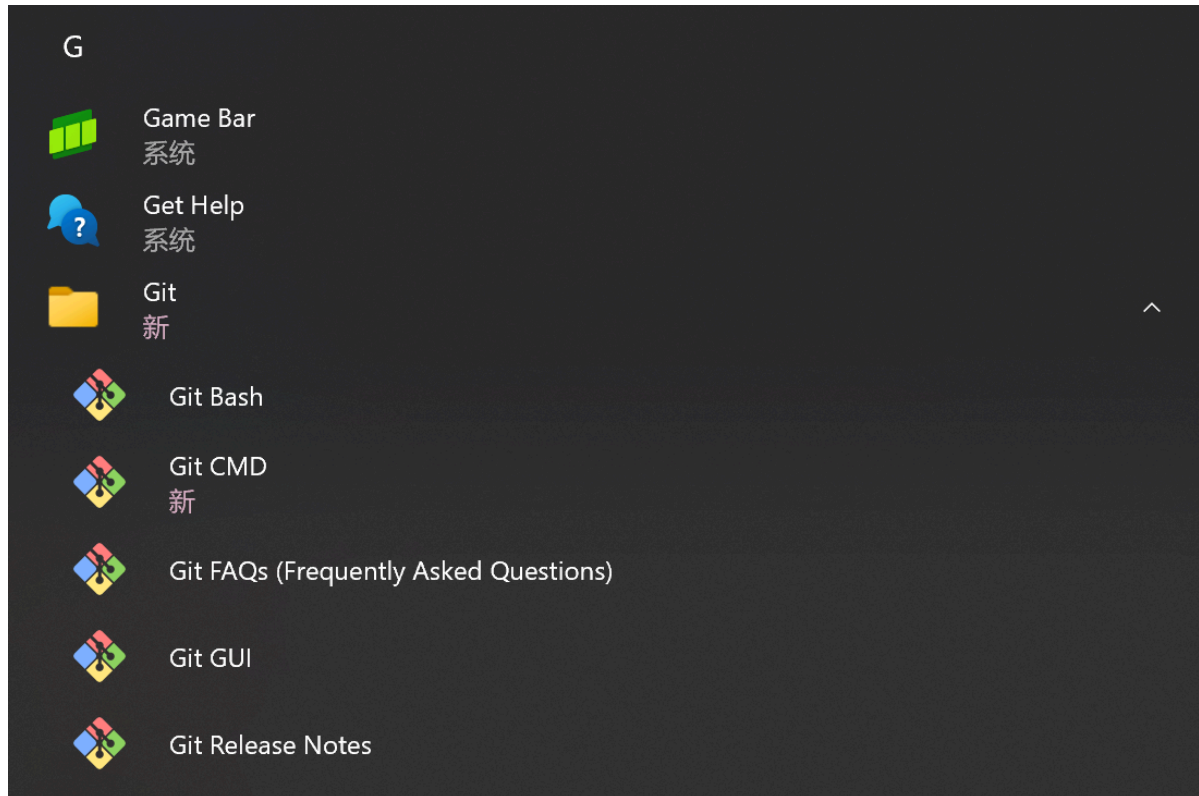
啊这个自己整吧hhh

在 Git 关联账号

啊其实也不能叫做绑定 Github 账号.....

只是填写一下个人信息，用于提交的时候显示你是谁而已，和 Github 账号没什么关联，只是有人喜欢让它们一致（比如我）

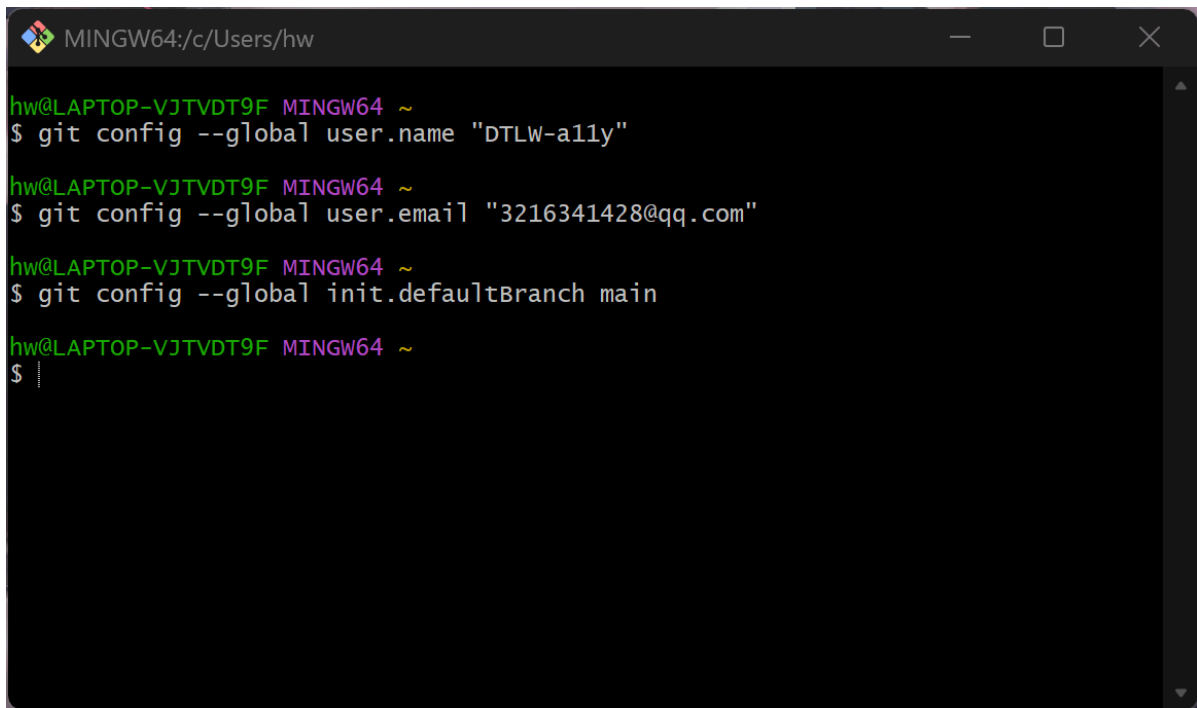
这是你安装好的Git：



打开 `Git Bash`，输入以下命令：

```
git config --global user.name "在双引号里填入你的用户名"  
git config --global user.email "在双引号里填入你的邮箱"  
git config --global init.defaultBranch main
```

不用在意为什么敲了回车一点反馈都没有。没有反馈就是没有问题！你Git是这样的！有反馈反而得小心咯。



```
MINGW64:/c/Users/hw
hw@LAPTOP-VJTVDT9F MINGW64 ~
$ git config --global user.name "DTLW-a11y"
hw@LAPTOP-VJTVDT9F MINGW64 ~
$ git config --global user.email "3216341428@qq.com"
hw@LAPTOP-VJTVDT9F MINGW64 ~
$ git config --global init.defaultBranch main
hw@LAPTOP-VJTVDT9F MINGW64 ~
$ |
```

这样就相当于登陆啦！以后再打开 Git Bash 的时候就不用再输一遍这三个代码了，它会用你这次输入的名字和邮箱进行提交代码。

试解 Git 原理

工作区

工作区是开发者直接编辑和操作代码的地方。它包含了项目的所有文件和子目录，是开发者日常工作的主要区域。

其实就是下文中，你为仓库而在你电脑本地建立的那个文件夹。

暂存区

暂存区是一个临时存放改动的区域。暂存区实际上是一个文件，保存即将提交的文件列表信息。

版本库

版本库是安全存放数据的位置，包含了所有提交的版本数据。

对于我们来说，这个版本库就是 Github 仓库。

提交流程

平常工作在 **工作区**，然后你想提交哪些会先记录在 **暂存区**，最后确认完毕，上传到 Github —— **版本库**，就可以所有人共享了。

Git 的使用

前言：

那个.....我都写完了才发现啦，其实 LFS 和 忽略规则 我自己这边整完就行，你们克隆下来以后就能用了，不需要你们再配置一遍，你们就当看个乐子吧（嘻）

Git Bash (喜欢敲命令嗷)

初来乍到

仓库建立

首先，在本地找一个合适的位置（比如我建立了个文件夹在D盘：`D:\Git`）

然后，打开 Git Bash，输入命令：

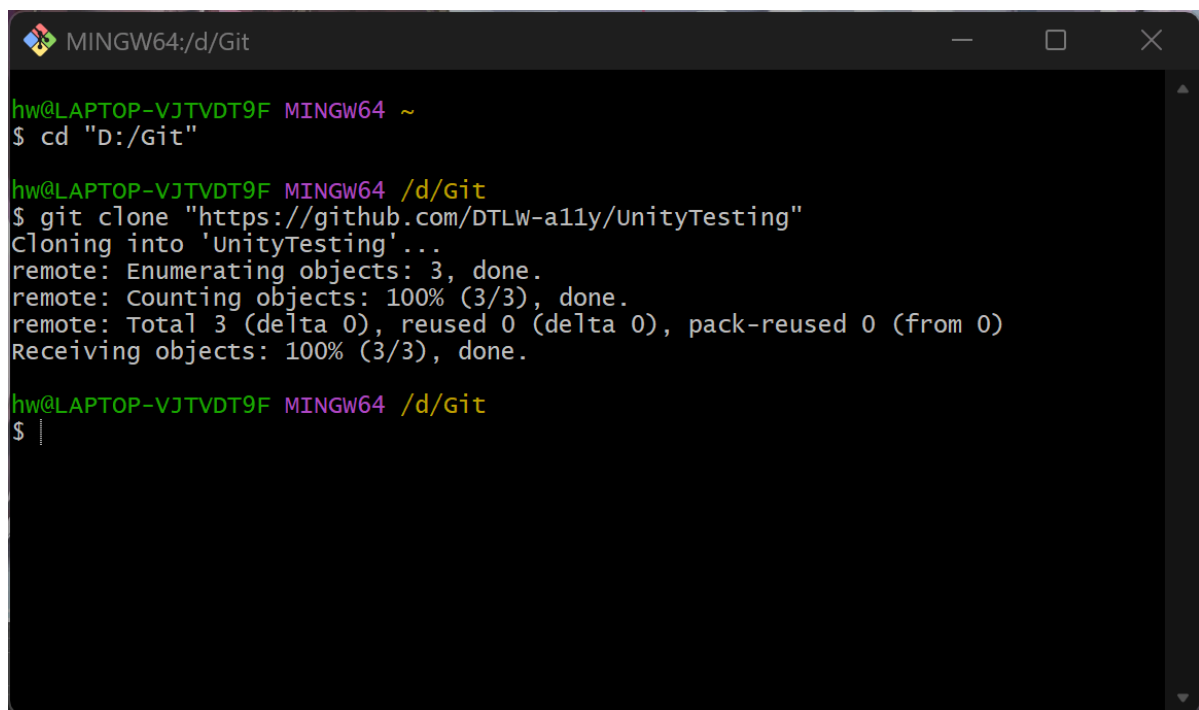
先定位：

```
cd "你的文件夹的路径"
```

然后克隆仓库：

```
git clone "Github仓库网址"
```

如下图：



```
MINGW64:/d/Git
hw@LAPTOP-VJTVDT9F MINGW64 ~
$ cd "D:/Git"

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git
$ git clone "https://github.com/DTLW-a1ly/UnityTesting"
Cloning into 'UnityTesting'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git
$ |
```

建立后

把Unity项目放进去吧！

怎么放呢？

举个例子：

假设，我在D盘创建了一个文件夹叫Git，进入Git执行git clone后，出现仓库
D:/Git/UnityProject1，然后我的已有Unity项目在C:/Desktop/Program文件夹中，怎么办呢？

正确的做法是 将已有Unity项目中的所有内容复制到克隆下来的仓库目录中：将 C:/Desktop/Program 文件夹内的所有内容（不包括 Program 文件夹本身）复制到 D:/Git/UnityProject1 中。

操作前

C:/Desktop/Program/ — Assets/ — ProjectSettings/ — Packages/ — (其他Unity项目文件)	D:/Git/UnityProject1/ — .git/ — README.md
---	--

操作后

D:/Git/UnityProject1/ — .git/ (保留的Git信息) — Assets/ (从Program复制过来) — ProjectSettings/ (从Program复制过来) — Packages/ (从Program复制过来) — (其他Unity项目文件) (从Program复制过来)
--

之后的编辑呢，可以选择在你原来的路径里去编辑，只是每一次提交的时候你都要复制并替换了。

也可以直接在Unity里打开新的项目路径，直接编辑！

进入仓库

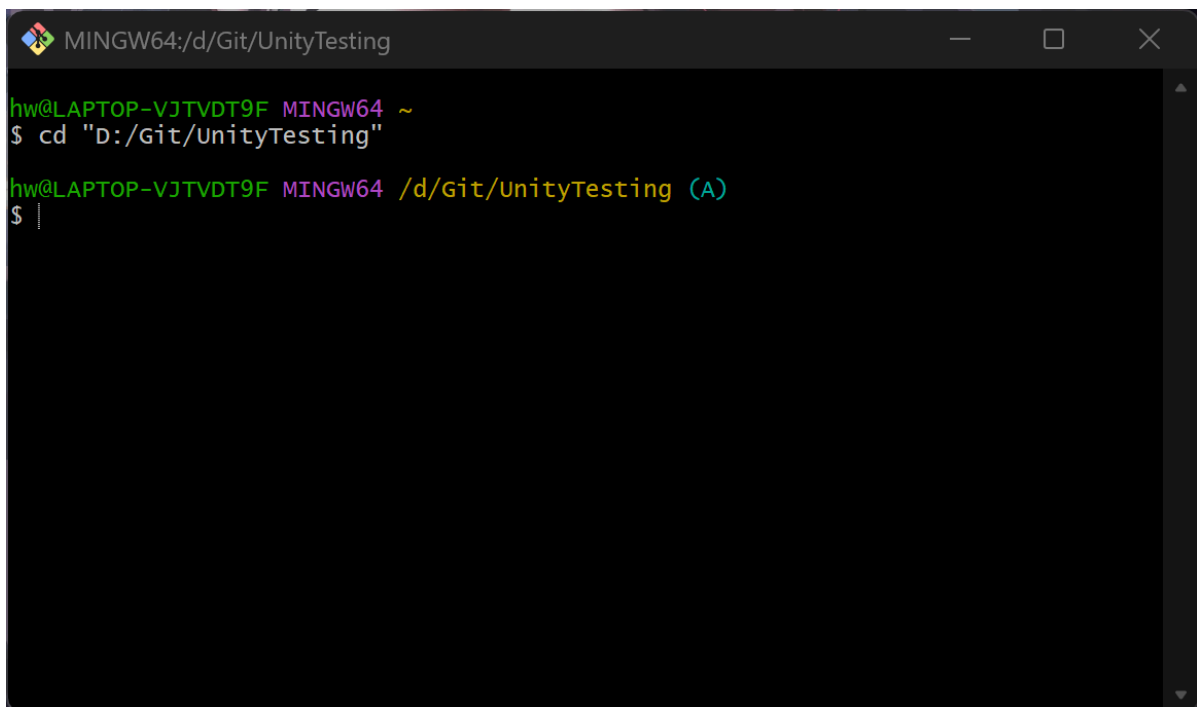
如上图，我克隆下来的仓库名字叫做 UnityTesting，你本地就会多出一个文件夹叫 UnityTesting。

找到这个文件夹路径，然后

```
cd "这个路径"
```

就可以进入仓库了。

如下图：



```
MINGW64:/d/Git/UnityTesting
hw@LAPTOP-VJTVDT9F MINGW64 ~
$ cd "D:/Git/UnityTesting"
hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (A)
$ |
```


LFS 配置

Git LFS (Large File Storage) 是一个用于管理 Git 仓库中大文件的扩展工具，对于处理音频、视频、图像、模型等二进制文件特别有用。

我这边看了下，好像总共是有10GB的免费额度的，然后一个月免费流量也是10GB（好像噻，毕竟网上有人说1GB，可我看的是10GB）。

如果不用 LFS 的话，那些很占空间的素材越堆越多，好像是会拖慢 Git 的速度，所以还是用一下为好罢。

初始化 LFS

先进入仓库吧。上一步应该搞好了。

然后

```
git lfs install
```

此命令会设置必要的 Git 钩子，使 Git LFS 能正常工作。

关联 LFS

这个务必要进入仓库里搞哦！

```
git lfs track "*.jpg"
git lfs track "*.jpeg"
git lfs track "*.png"
git lfs track "*.webp"
git lfs track "*.bmp"
git lfs track "*.tiff"

git add .gitattributes
git commit -m "配置Git LFS跟踪规则：图片"
git push origin main
```

就先简单追踪一下图片吧，毕竟我们好像也没有其他的了.....

有再加嘛！

（关于 push 这一块，我们先放在 main 里面吧。刚开始主要是共享代码，先放在 main。等我们所有人都下下来了，各自开工后就自己放在自己的分支吧。）

（这些分支什么的，没看懂没关系，紧接下来就会提到了！）

使用小技巧

- **查看已跟踪的模式：**如果你想确认当前项目都设置了哪些跟踪规则，直接在项目目录下运行

```
git lfs track
```

- **检查将被跟踪的文件：**查看哪些文件已经被 Git LFS 管理，使用

```
git lfs ls-files
```

临时文件忽略

临时文件通常非常巨大，而它们通常可以通过源代码再次生成，并且也没有必要对于这些临时的缓存进行版本管理，所以我们有必要设置一些忽略规则！

但是，已经有官方帮我们编写好规则啦！

只要下载好了规则文件（.gitignore），然后放在仓库的根目录里（比如我之前所有图片里的例子，仓库叫做 UnityTesting，路径为 D:/Git/UnityTesting，那就放在这个 UnityTesting 里面），然后

```
git add .gitignore
git commit -m "更新忽略规则：添加新的临时文件类型"
git push origin main
```

（关于 push 这一块，我们先放在 main 里面吧。刚开始主要是共享代码，先放在 main。等我们所有人都下下来了，各自开工后就自己放在自己的分支吧。）

（重要的事情说两遍）

日常使用

初始化

每次打开 Git Bash 的时候，要记得做一点前提准备。

- 先定位到你的仓库。

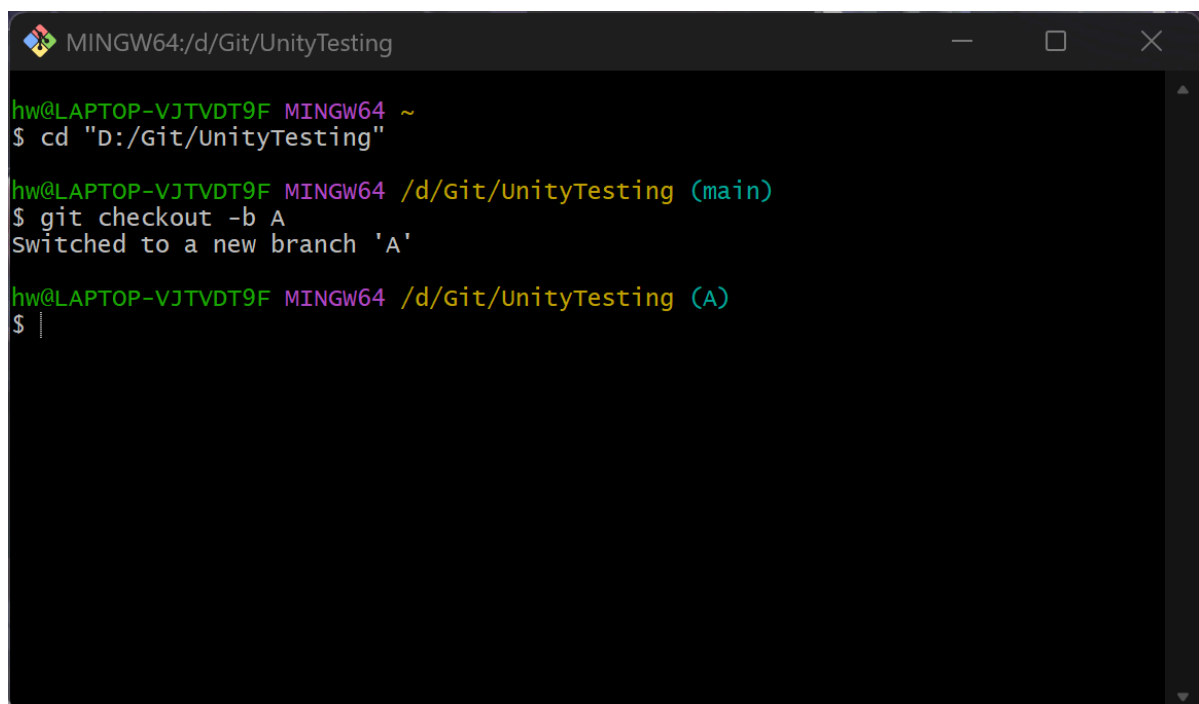
```
cd "仓库路径"
```

然后你就能看到，黄色字是当前路径，青色字是当前分支。

- 切换分支（默认是 main 我记得）

```
git checkout -b 分支名
```

栗子：



```
MINGW64:/d/Git/UnityTesting
hw@LAPTOP-VJTVDT9F MINGW64 ~
$ cd "D:/Git/UnityTesting"

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$ git checkout -b A
Switched to a new branch 'A'

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (A)
$ |
```

提交代码

分三步：

1. 把你本地的有过改动的文件刷进缓存区。

一般用让它自动检测，把目录下所有有过修改的文件全部刷进去。

```
git add .
```

"." 的意思就是任意名字、任意后缀，即所有文件。

也可以指定文件名或指定后缀名。

豪用！

2. 把缓存区提交到本地仓库（我也不懂这个本地仓库是个啥 🤔🤔 反正影响不大，也必不可少）。

```
git commit -m "你对于这次提交想说的话，什么 版本啊，内容啊，特性啊"
```

3. 上传到远程仓库。即提交到 Github 上去。

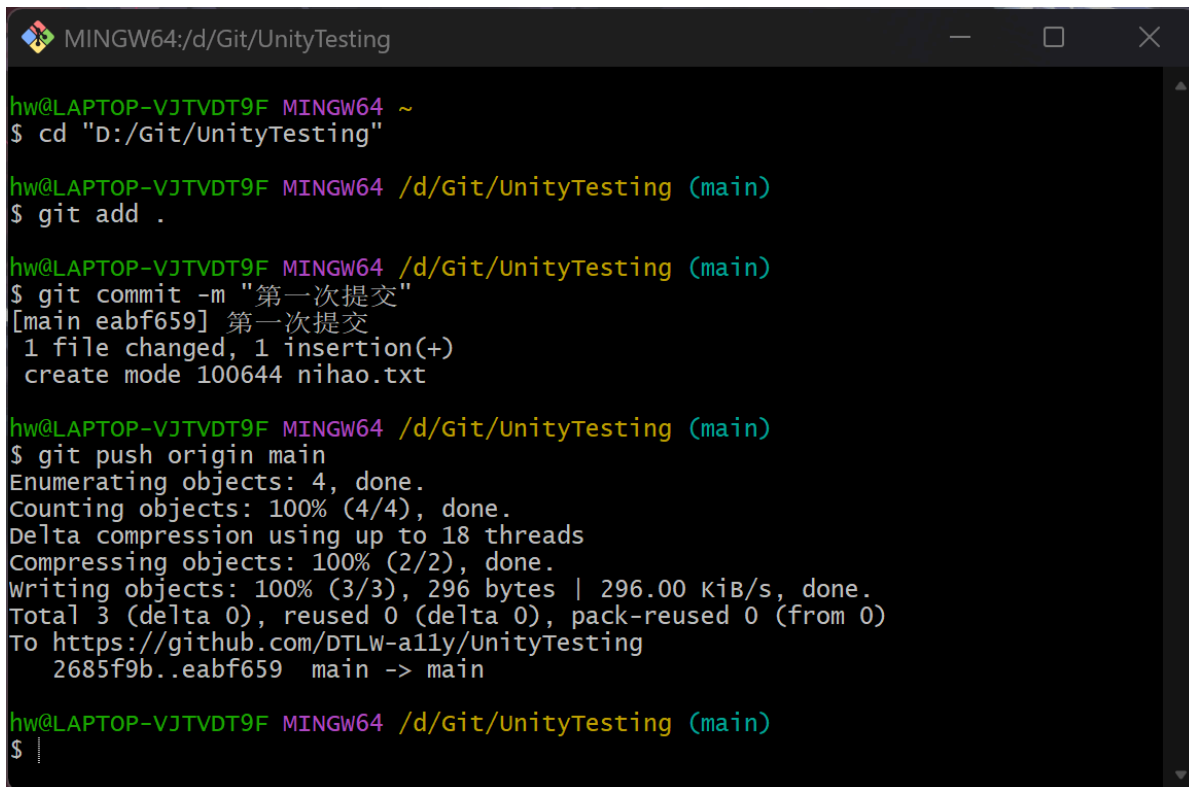
```
git push origin 分支名
```

至于这个分支，就可以理解为仓库里的不同文件夹。

默认的，自带的，最大的，显示在 Github 首页的，就是 main 分支。

可以一个人专门一个分支，每个人提交代码就提交到自己的分支，最后再来合并分支。

举个栗子：

A screenshot of a Windows terminal window titled "MINGW64:/d/Git/UnityTesting". The terminal shows a series of git commands and their outputs. The user navigates to the directory "D:/Git/UnityTesting", adds the current directory to the staging area, commits the changes with the message "第一次提交", and then pushes the changes to the 'main' branch of the remote repository 'origin'. The output shows the commit hash [main eabf659] and the push progress, including object counting and compression. The final output shows the push was successful, creating a new commit 2685f9b on the main branch.

```
MINGW64:/d/Git/UnityTesting
hw@LAPTOP-VJTVDT9F MINGW64 ~
$ cd "D:/Git/UnityTesting"

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$ git add .

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$ git commit -m "第一次提交"
[main eabf659] 第一次提交
1 file changed, 1 insertion(+)
create mode 100644 nihao.txt

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 18 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 296 bytes | 296.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/DTLW-a1ly/UnityTesting
2685f9b..eabf659  main -> main

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$ |
```

撤销提交

1. 先看看提交历史：

```
git log --oneline
```

它就会显示出你的提交历史，前面黄色的字母相当于是对应的编号，后面就是你的提交时的留言。

选择一下你要撤销哪一个。

撤销操作也时可以撤销的，撤销你的撤销。

2. 开始撤销：

```
git revert 那一串编号
```

然后会弹出文本编辑器（比如 VScode）让你给这次撤销操作留言。一般可以直接关掉其实。

3. 同步到远程仓库：

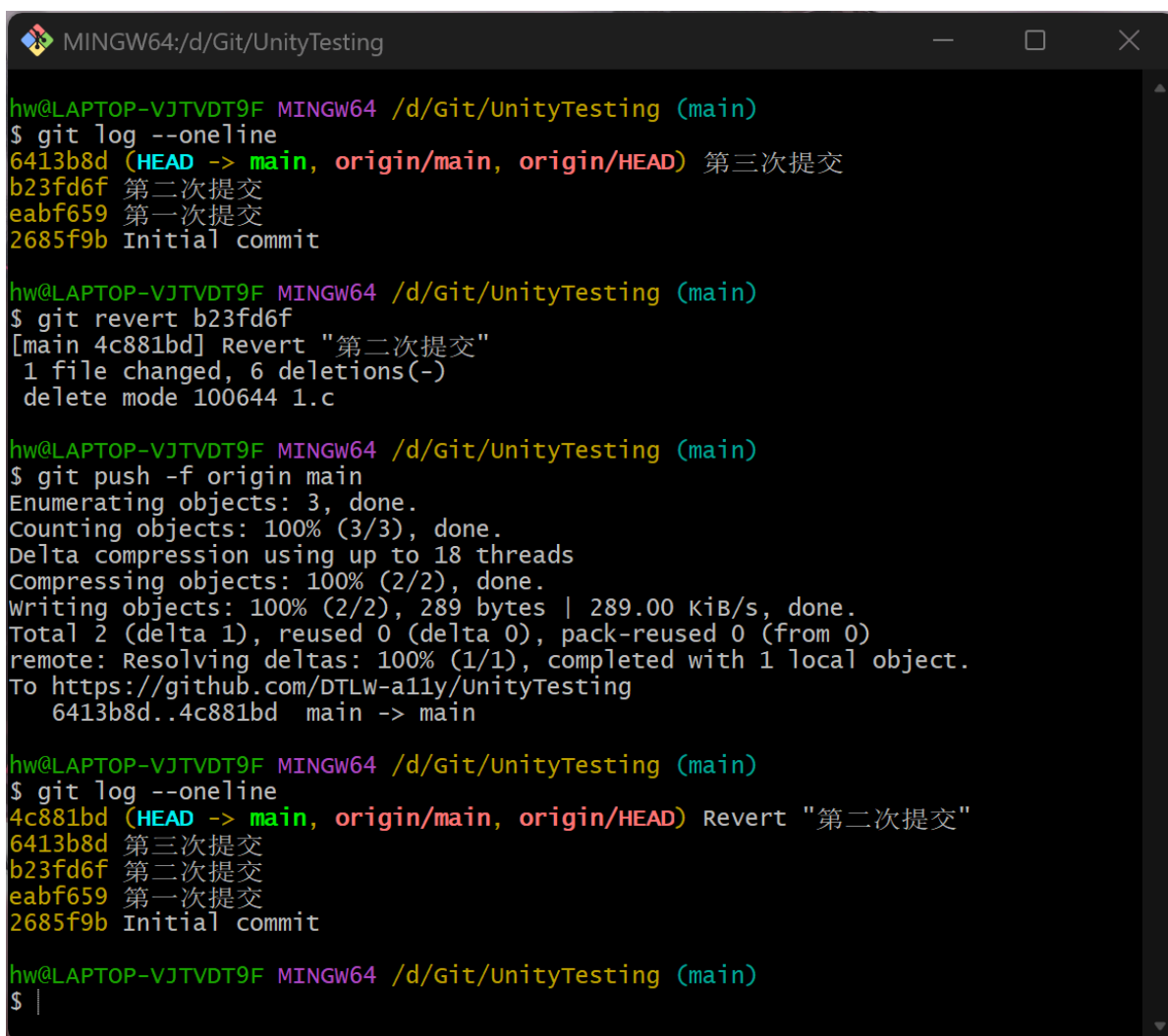
```
git push -f origin 分支名
```

这个时候，就可以把你的撤销同步到 Github 上了。

注意！这些操作会同步修改你的工作区的！所以，如果有需要的话，记得备份！

没备份也没事，可以撤销你的撤销的。

照样是例子：



```
MINGW64:/d/Git/UnityTesting

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$ git log --oneline
6413b8d (HEAD -> main, origin/main, origin/HEAD) 第三次提交
b23fd6f 第二次提交
eabf659 第一次提交
2685f9b Initial commit

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$ git revert b23fd6f
[main 4c881bd] Revert "第二次提交"
1 file changed, 6 deletions(-)
delete mode 100644 1.c

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$ git push -f origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 18 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 289 bytes | 289.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/DTLW-a1ly/UnityTesting
6413b8d..4c881bd main -> main

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$ git log --oneline
4c881bd (HEAD -> main, origin/main, origin/HEAD) Revert "第二次提交"
6413b8d 第三次提交
b23fd6f 第二次提交
eabf659 第一次提交
2685f9b Initial commit

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$ |
```

清空缓存区

```
git reset
```

上述撤销提交是针对已经交到仓库里去了的。

而这个是清空缓存区，也就是你用了 add，但是还没用 commit。

有时候 add 错文件了，就可以这样清除，然后重新来过。

清空 Git Bash 界面

```
clear
```

是的没错就这么简单，重获干净。

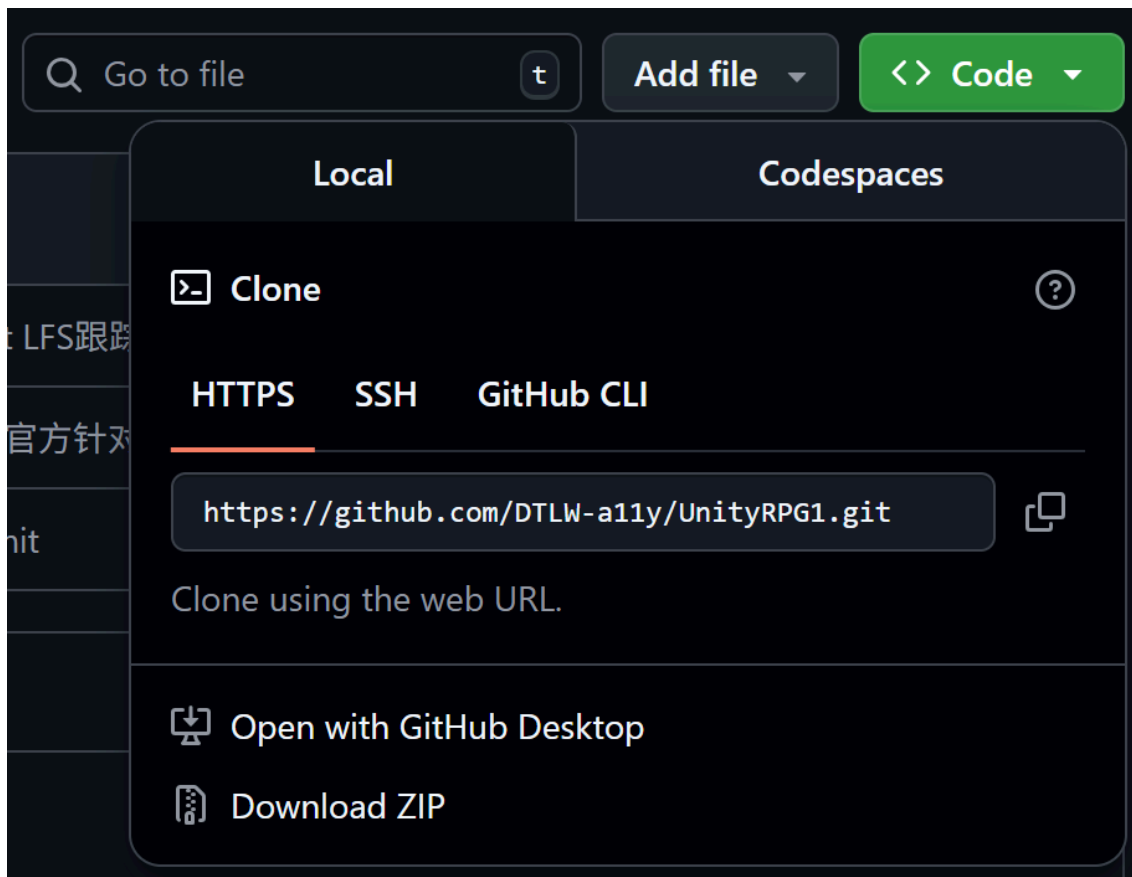
下载代码

比如现在，我要把A分支里面的代码下载到本地，有两种方案：

1. Github 下载

从 Github 网站上下载代码的 ZIP 压缩包，自行解压放到本地仓库中。

仓库界面 --> Code --> Download ZIP



这种方法方便快捷，**但是!!!** 不能下载 LFS 里面的东西。所以会出现一些情况，比如：资源包丢失。

2. 使用 Git Bash

```
git checkout A
git reset --hard origin/A
```

注意！这一块代码执行完后，会强制把你本地仓库里的文件替换成你下载下来的东西，也就是说：**你自己的东西会没掉，换成下载的。**所以，要想好来，或者是自己备份。

而以下的两行代码是完整拉取 LFS 中文件的命令，就是手动再下载一遍 LFS 中的文件，确保 LFS 中的资源文件一定被下载。

```
git lfs fetch --all
git lfs pull
```

怎么说呢，一般不用这两句话的。如果你用了第一块代码后，资源包还是用不了，就可以重新删掉再下载，然后加上这第二块代码。

合并代码

基本格式：

```
git merge 分支名
```

就是把你填入名字的那个分支合并到你的当前分支来。

如下图，一个很长的例子（分别将 A、B 合并进入 main 分支中，而 A、B 自身不变）：

```
MINGW64:/d/Git/UnityTesting

hw@LAPTOP-VJTVDT9F MINGW64 ~
$ cd "D:/Git/UnityTesting"

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$ git merge A
CONFLICT (modify/delete): Add.cpp deleted in HEAD and modified in A.  Version A
of Add.cpp left in tree.
Automatic merge failed; fix conflicts and then commit the result.

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main|MERGING)
$ git merge B
error: Merging is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main|MERGING)
$ git add .

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main|MERGING)
$ git commit -m "merge A"
[main 97a9ddb] merge A

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$ git merge B
Auto-merging Add.cpp
CONFLICT (content): Merge conflict in Add.cpp
Automatic merge failed; fix conflicts and then commit the result.

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main|MERGING)
$ git add .

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main|MERGING)
$ git commit -m "merge B"
[main aabda3f] merge B

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$ git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 18 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 665 bytes | 665.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/DTLW-a1ly/UnityTesting
   a9cb8ac..aabda3f  main -> main

hw@LAPTOP-VJTVDT9F MINGW64 /d/Git/UnityTesting (main)
$
```

主要就是：每次 merge 完以后，如果你处于图中所示的（MERGING）状态，你就要去本地检查一遍看看代码有没有问题（比如有无法自动解决的冲突的地方，git 会给你标记出来，你要手动修改），检查、修改，得到结果后进行 add、commit 后，才可以继续下一个合并。